FAQ for judges

Fifteen questions a judge is likely to open with, and the short answer to each. Two or three sentences, then the file or command that backs it. The full, source-linked list is docs/DEFENSE_FAQ.md; the numbers are in docs/METRICS.md and docs/REAL_CORPUS_ANALYSIS.md.

1. Why not use Wireshark to do the parsing?

Wireshark is a program, not a library, so using it would mean shelling out to tshark and depending on an external binary. dpkt reads both PCAP and PCAPNG in pure Python, so the whole tool installs from pip alone and runs offline. See ARCHITECTURE.md, src/capture/.

2. What if the protocol is encrypted?

It is out of scope: the tool reports bytes as they appear and makes no claim about their meaning. It still lists the TCP sessions and the byte streams, so the envelope is visible even when the payload is not. See the limits table in README.md, docs/SUBMISSION.md.

3. How do you check correctness on new data?

The rule is applied unchanged to a capture it was not fitted to, and every message is classified. Rule v2 transfers to corpus_capture_02 with 40 matched and 0 counterexamples, and fails openly on synthetic_live. See REPORT.md sections 7-8, tests/integration/test_reference_report.py.

4. Why 794 tests, and not more?

The suite covers every module and every status the engine can emit, including the boundary and defect fixtures; coverage is 90 % over src/, and the remaining lines are error branches that need a real I/O failure. More tests would not raise the claim, which is "confirmed in scope", not "proven". See docs/AUDIT.md sections 4 and 11. The figure is the count at the archived v1.0.0 submission; the working main has since grown to 957.

5. What happens on a conflicting overlap?

The reassembler keeps the first bytes, records an ambiguity diagnostic with both versions, and a field that covers the overlap is reported ambiguous, not matched. Nothing is silently overwritten. See tests/protocol/test_engine.py::test_ambiguity_in_field_range_is_ambiguous.

6. Why dpkt and not PyShark?

PyShark shells out to tshark, which would have to be present on the target machine; the reference environment does not guarantee it. dpkt has no external process and is fast enough for 250k packets. See ARCHITECTURE.md.

7. What is provenance and why does it matter?

Every byte of a reassembled stream carries the packet it came from: index, sequence, timestamp and offset. That makes each verdict checkable against the pcap instead of merely plausible. See src/capture/provenance.py, docs/GUI_VERIFICATION.md.

8. How does a counterexample work?

A counterexample is a `mismatched` message bound to its bytes, its offset and its source packet, kept in the report rather than dropped. The eighty v1 counterexamples on `corpus_capture_01` are what widened the rule to v2. See `docs/CORPUS_ANALYSIS.md` section 4, `docs/RULE_V1_V2.md`.

9. What did you not do, and why?

No `B_to_A` field-by-field interpretation in the shipped demo rule, and no live-device capture; each is stated as a limit, not hidden. The window's Project menu covers the portable project, and the engine honours a `B_to_A` scope, but the demo rule scopes only `A_to_B`. The organizer set was generated framing over real application bytes, so a passive capture is the next check. See `REPORT.md` section 12, `docs/REAL_CORPUS_ANALYSIS.md` section 9.

10. How do you guarantee reproducibility?

Every figure comes from a shipped file and a named command, and the environment is pinned in `pyproject.toml` (Python 3.12+, fixed dependency versions). The project stores relative paths and sha256 digests, so it reopens unchanged on another machine. See `docs/PORTABILITY.md`, `docs/CONTRACT.md`.

11. On what data did you check?

Two corpora: a synthetic one (`tests/corpus/`, three captures, 369 framed messages) and the organizer set (`tests/corpus/real/`, four research captures plus eight diagnostic captures, 45 requests + 45 responses). See `docs/CORPUS_ANALYSIS.md`, `docs/REAL_CORPUS_ANALYSIS.md`.

12. What if a PCAP file is damaged?

A damaged but recognizable file is read as far as possible; the rest becomes diagnostics. Truncated headers, unsupported link types, incomplete fragments and mixed link types are reported, not raised. A fragmented datagram is reassembled when its bytes are all present. See `docs/METRICS.md` (Robustness), `tests/fixtures/defects/`.

13. How does it scale to 100k messages?

The reference profile (250000 packets, 1000 sessions, 95.66 MiB) parses, reassembles and exports in 12.10 s and peaks at 350 MiB against a 4 GiB budget. Streaming mode lowers the peak to 224 MiB. See `docs/BENCHMARK.md`, `docs/METRICS.md`.

14. What if a rule is wrong?

The wrong rule is kept, not deleted: it stays with its version, its result is marked `outdated`, and the counterexamples it produced are the evidence for the next version. A reader can always tell which rule produced which verdict. See `src/hypothesis/versioning.py`, `tests/hypothesis/test_versioning.py`.

15. How do you move the research to another machine?

The investigation is a portable directory: relative paths plus sha256 digests, exportable and importable as a zip with integrity checking. A clean unpack runs the whole suite (793 passed, 1 skipped from the published archive). See `docs/PORTABILITY.md`, `src/project/`.

Where to look next

- Numbers with their source: `docs/METRICS.md`.
- The long question list: `docs/DEFENSE_FAQ.md`.
- The pre-flight list: `docs/DEFENSE_CHECKLIST.md`.